Algorithmization and Programming of Solutions

Part I

TOPIC #5

CONTROL STATEMENTS IN PROGRAMMING LANGUAGE C. INPUT AND OUTPUT FUNCTIONS

*RTU LECTURER **OLGA YAKOVLEVA**, MG.SC.ING*

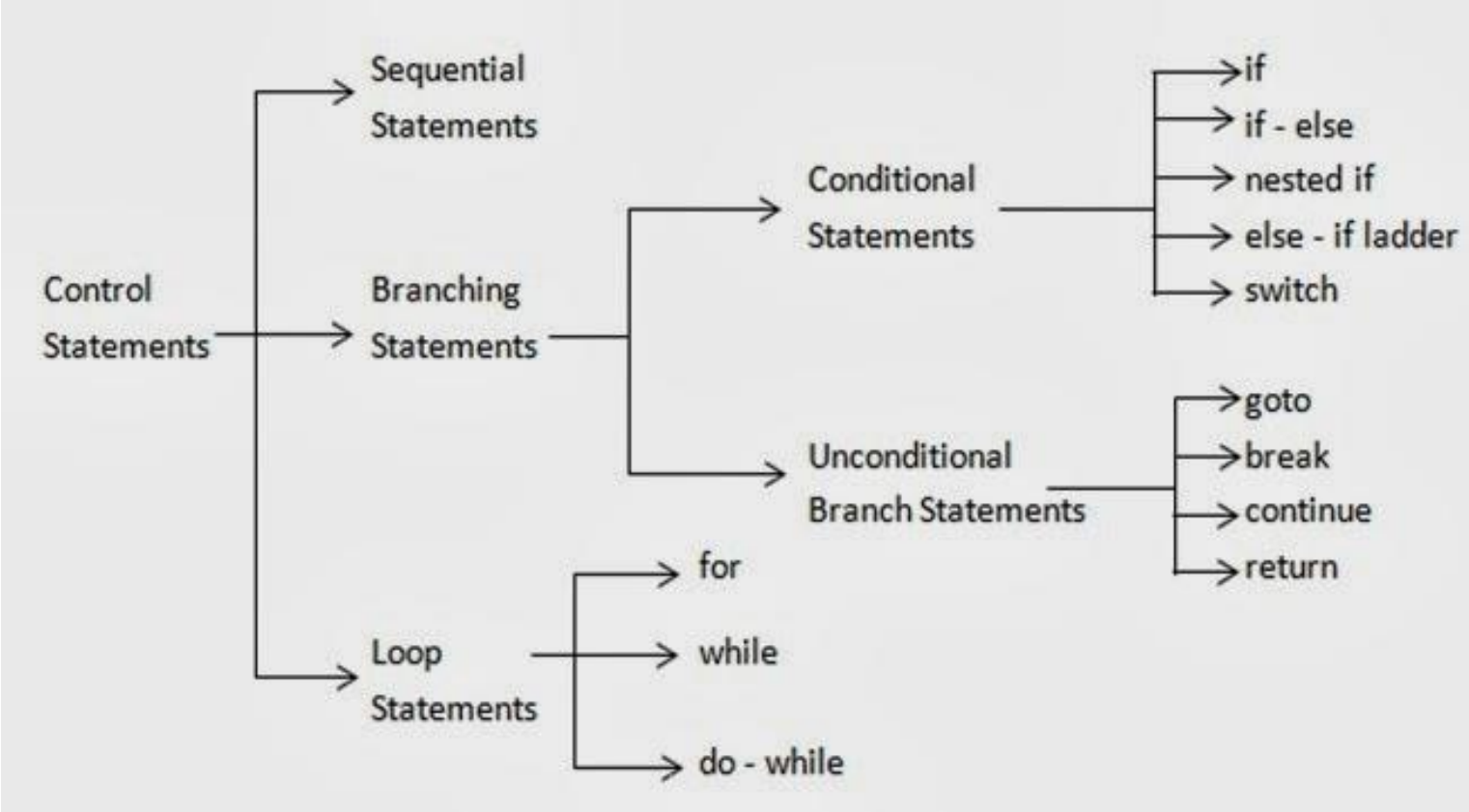
Table of contents

1. Control statements in programming language C

IF...ELSE, SWITCH...CASE, GOTO, WHILE, DO...WHILE, FOR, BREAK, CONTINUE

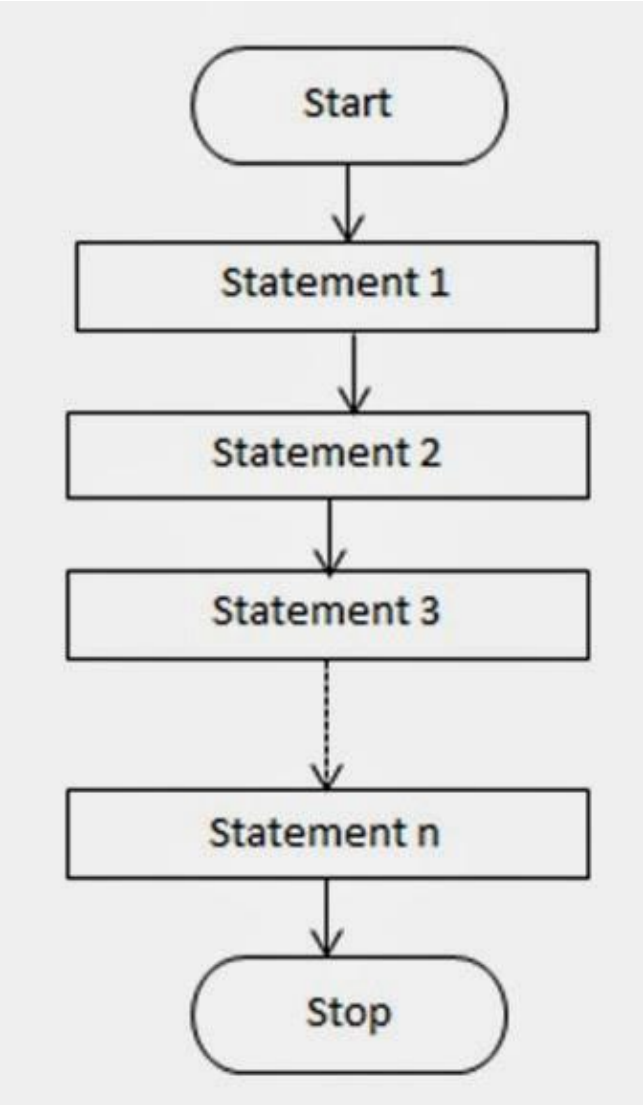
Classification of control statements

Picture original source http://dotprogramming.blogspot.com/2013/11/c-programming-Uses-of-Sequential-and-Compound-Statements.html



Sequential statements

Picture original source http://dotprogramming.blogspot.com/2013/11/c-programming-Uses-of-Sequential-and-Compound-Statements.html



Braces

Braces { } are used to group several actions, when syntax allows to use only one action in statement, e.g.:

```
// if there are no braces, if condition relates to the first action only
if (x == 1)
   a = 0;
   b = 5;
```

```
// if there are braces, if condition relates to all actions putted in braces
if (x == 1) {
   a = 0;
   b = 5;
}
```

Conditional branching statements

IF statement

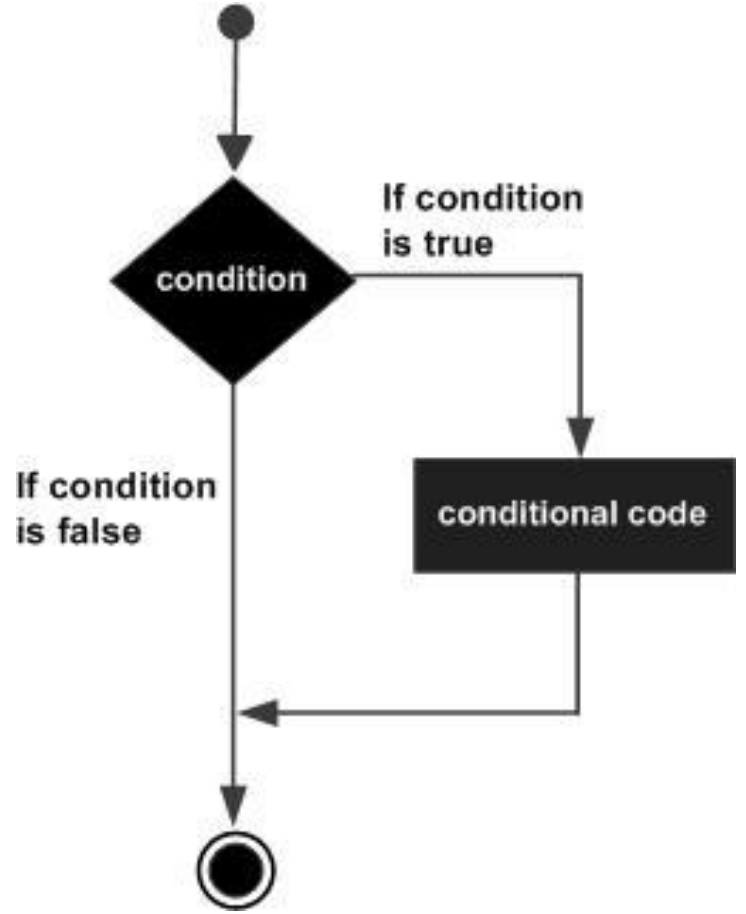
Picture original source http://www.tutorialspoint.com/cprogramming/if_statement_in_c.htm

General form:

- `if (condition) operator1`

Example:

- `if (a>b) c=a/b;`



Conditional branching statements

IF .. ELSE statement

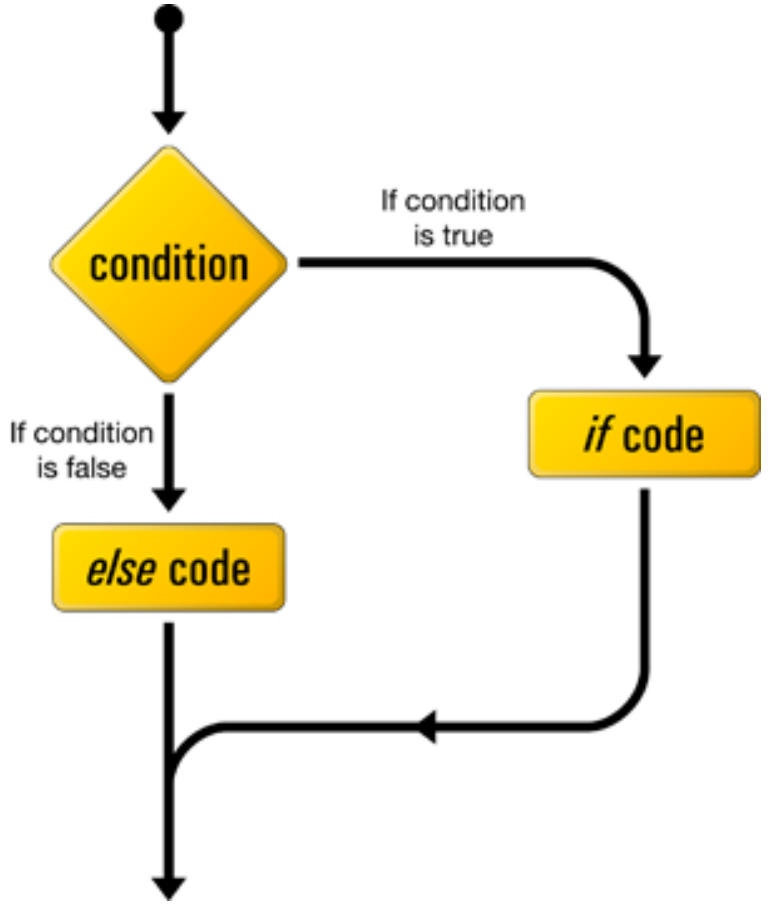
Picture original source http://byterevel.com/2011/07/13/objective-c-tutorial-3-if-else-statements-and-basic-looping/

General form:

- `if (condition) operator1 else operator2`

Example:

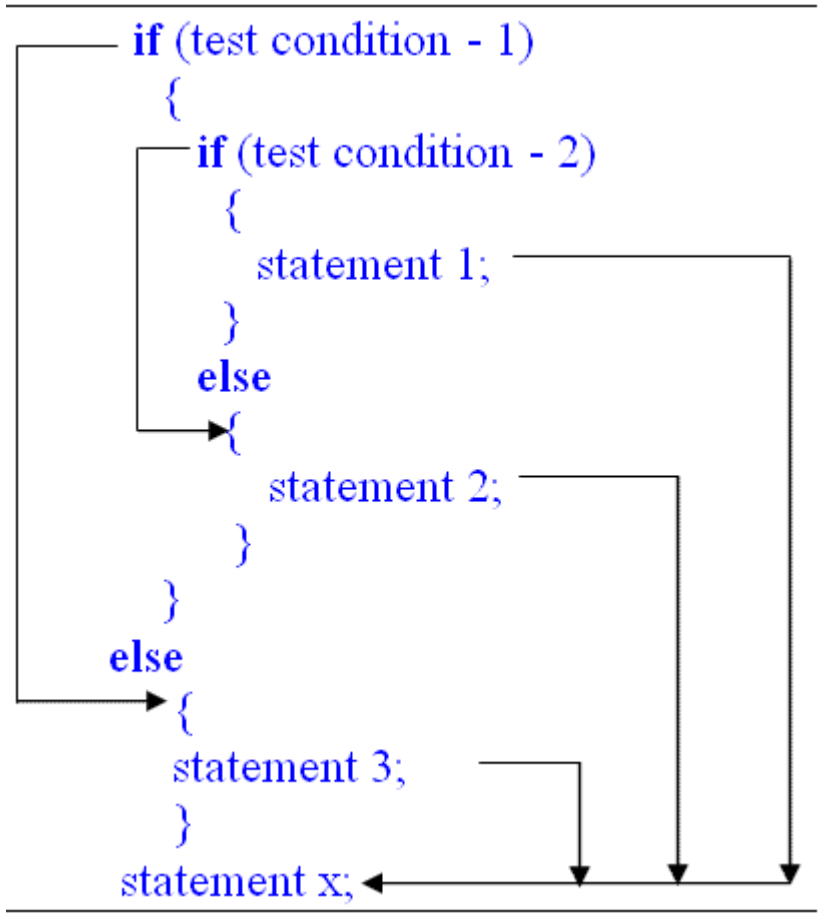
- `if (a>b) c=a/b; else c=b/a;`
- `c=(a>b)?a/b:b/a;`



Conditional branching statements

Nested IF, general form

Picture original source http://tanmayonrun.blogspot.com/2011/06/c-programming-decision-making-and.html



Conditional branching statements

Nested IF, example (1/2)

```
#include <stdio.h>

int main () {

   int a = 100, b = 200;

   if( a == 100 ) {

   /* if condition is true then check the
    following */

      if( b == 200 ) {

      /* if condition is true then print the
        following */
```

Conditional branching statements

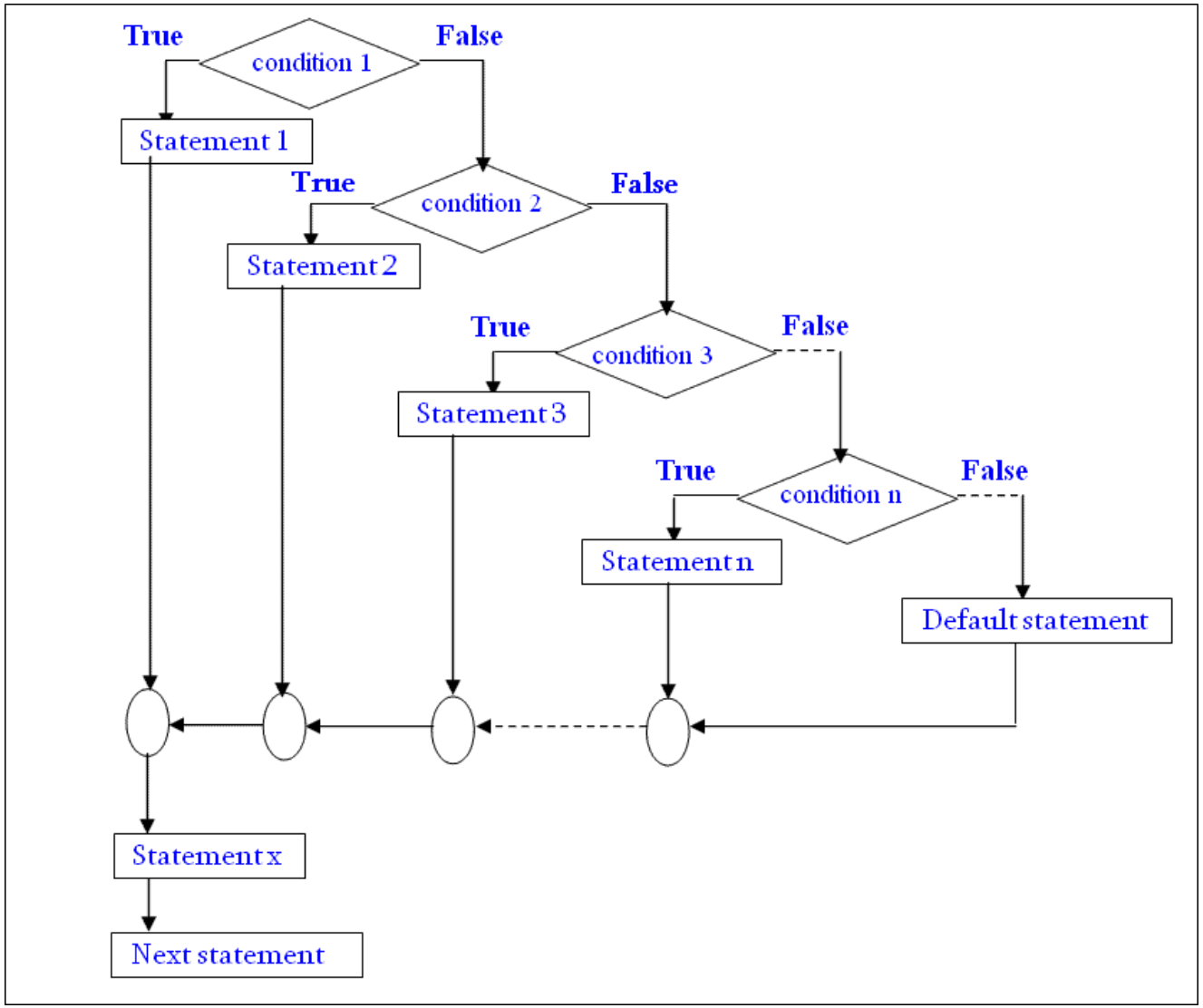
Nested IF, example (2/2)

```
        printf("Value of a is 100 and b is
                200\n");

        }

        else printf("Exact value of b is : %d\n",
                    b);

    }

    else printf("Exact value of a is : %d\n", a);

    return 0;

}
```

Conditional branching statements

ELSE IF ladder, general form

Picture original source http://tanmayonrun.blogspot.com/2011/06/c-programming-decision-making-and.html



Conditional branching statements

ELSE IF ladder, example (1/2)

```
#include <stdio.h>
int main() {
   int age;
   printf("Please enter your age");
   scanf("%d", &age);
   if ( age < 100 )
     printf ("You are pretty young!\n");
```

Conditional branching statements

ELSE IF ladder, example (2/2)

```
  else if ( age == 100 )
    printf("You are old\n");
  else
    printf("You are really old\n");
  return 0;
}
```

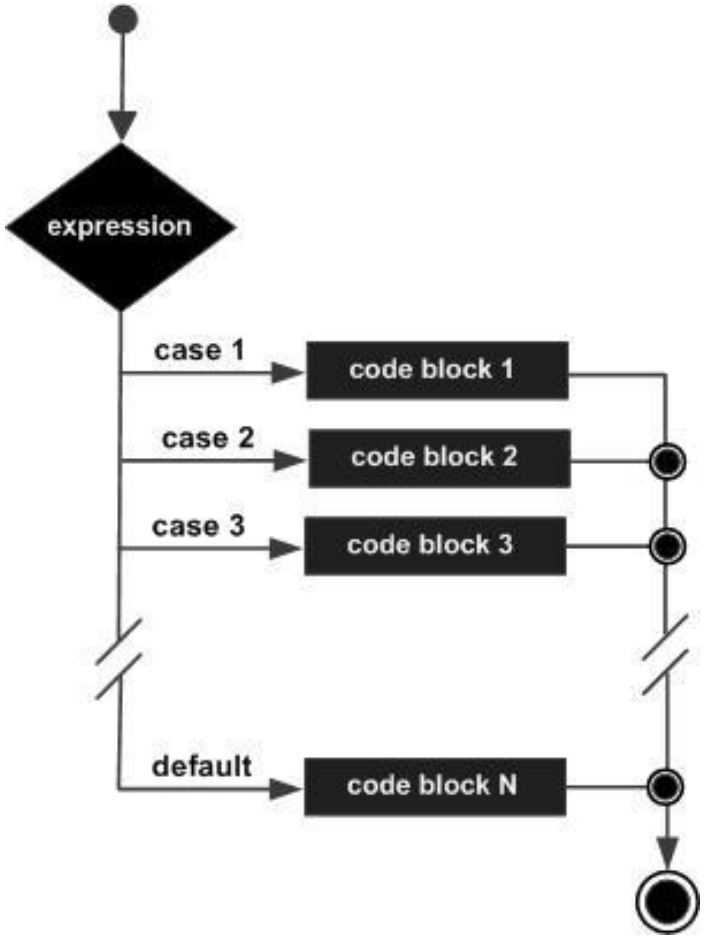
Conditional branching statements

SWITCH .. CASE statement, general form

Picture original source http://www.tutorialspoint.com/cprogramming/switch_statement_in_c.htm

General form:

```
switch(expression) {
   case expression :
      operator1;
   ...
   default :
      operatorN;
}
```



Conditional branching statements

SWITCH .. CASE statement, example (1/2)

```
#include <stdio.h>
int main () {
 char grade;
 printf("Input grade: ");
 scanf("%c", &grade);
 switch(grade) {
  case 'A' :
   printf("Excellent!\n"); break;
  case 'B' : case 'C' :
   printf("Well done\n"); break;
```

Conditional branching statements

SWITCH .. CASE statement, example (2/2)

```
  case 'D' :
   printf("You passed\n"); break;
  case 'F' :
   printf("Try again\n"); break;
  default :
   printf("Invalid grade\n");
 }
 return 0;
}
```

Loop statements

FOR statement, general form

Picture original source http://www.tutorialspoint.com/cprogramming/c_for_loop.htm

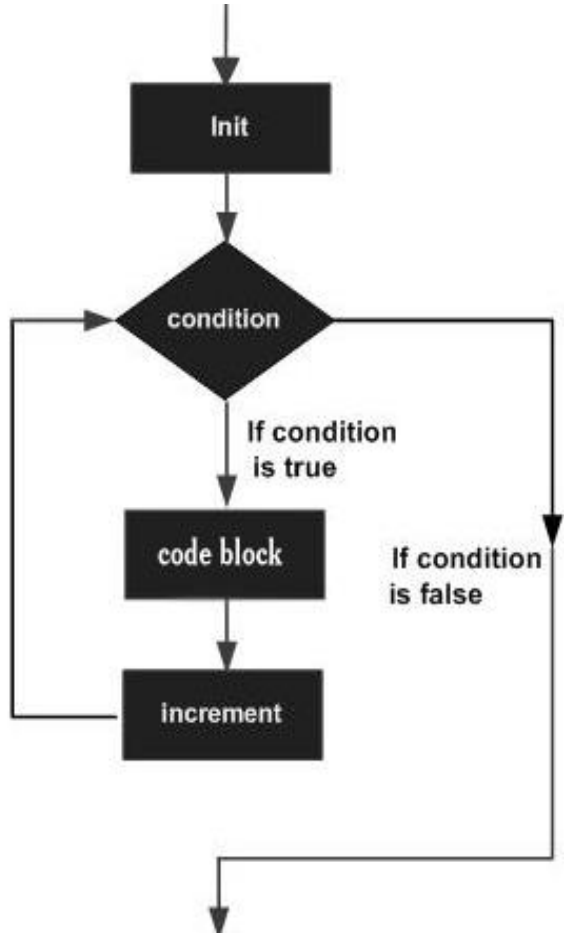
General form:

- **for** (initialization; condition; increment)
  operator

Example:

- **for** (a=0; a<10; a++)
  printf("%d", a);

- **for** (a=10; a<0; a--)
  printf("%d", a);



Loop statements

FOR statement, example (1/2)

```
#include <stdio.h>
int main() {
   int n, count, sum=0;
   printf("Enter the value of n.\n");
   scanf("%d",&n);
   for (count=1;count<=n;++count) {
      sum+=count;
   }
```

Loop statements

FOR statement, example (2/2)

```
    printf("Sum=%d",sum);
    return 0;
}
```

Loop statements

FOR statement, other examples (1/2)

```
int i;
 // infinite loop
 for ( ; ; ) printf("%d", i);
```

```
int i;
 // infinite loop
 for ( ; ; i++) printf("%d\t", i);
```

```
0       1       2       3       4       5       6       7       8       9
10      11      12      13      14      15      16      17      18      19
20      21      22      23      24      25      26      27      28      29
30      31      32      33      34      35      36      37      38      39
40      41      42      43      44      45      46      47      48      49
50      51      52      53      54      55      56      57      58      59
```

Loop statements

FOR statement, other examples (2/2)

```
int i;
 // finite loop
 for ( ; i<5; i++) printf("%d\t", i);
```

```
0	1	2	3	4
```

```
int i;
 // finite loop with unusual increment
 for (i=3; i<100; i=i*2) printf("%d\t", i);
```

```
3	6	12	24	48	96
```

Loop statements

WHILE statement, general form

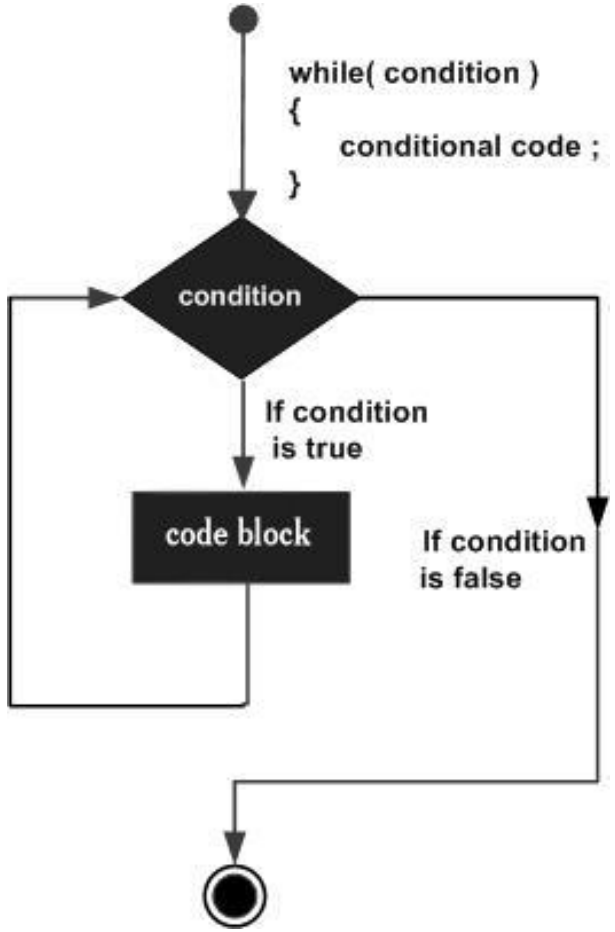
Picture original source http://www.tutorialspoint.com/cprogramming/c_while_loop.htm

General form:

- **while** (condition) operator

Example:

- **while** (a>0) a--;
- **while** (a>0) {
  printf("%d", a);
  a--;
  }



Loop statements

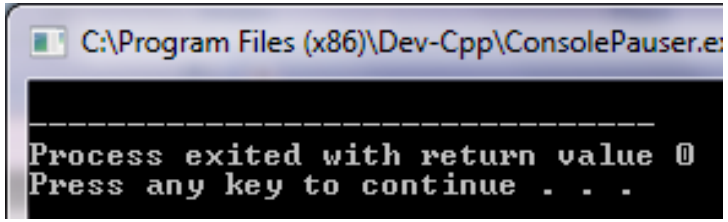
WHILE statement, program example

```
#include <stdio.h>
int main() {
   int x = 0;
   while ( x < 10 ) {
      printf("%d\n", x);
      x+=2;
   }
   getchar();
}
```

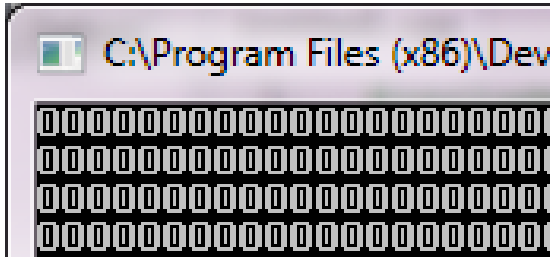
Loop statements

WHILE statement, other examples (1/2)

```
int i;
 // finite loops
 (will not execute at all)
 while (0) printf("%d", i);
 while (false) printf("%d", i);
```




```
int i;
 // infinite loops
 while (1) printf("%d", i);
 while (true) printf("%d", i);
```



Loop statements

WHILE statement, other examples (2/2)

```
int i=-5, j=3;
 // finite loop
 while (i++ && j--)
    printf("i = %d, j = %d\n", i, j);
```

Loop statements

DO .. WHILE statement, general form

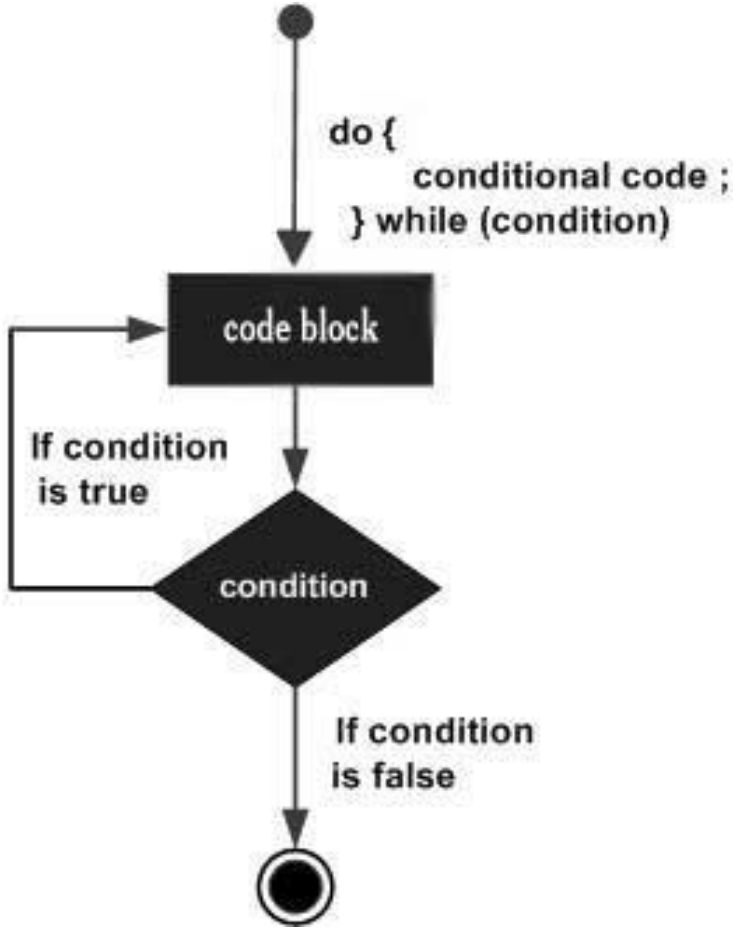
Picture original source http://www.tutorialspoint.com/cprogramming/c_do_while_loop.htm

General form:

- **do** operators
  while (condition)

Example:

- **do** a--;
  while (a>0);
- **do** {
  printf("%d", a);
  a--;
  } **while** (a>0);



Loop statements

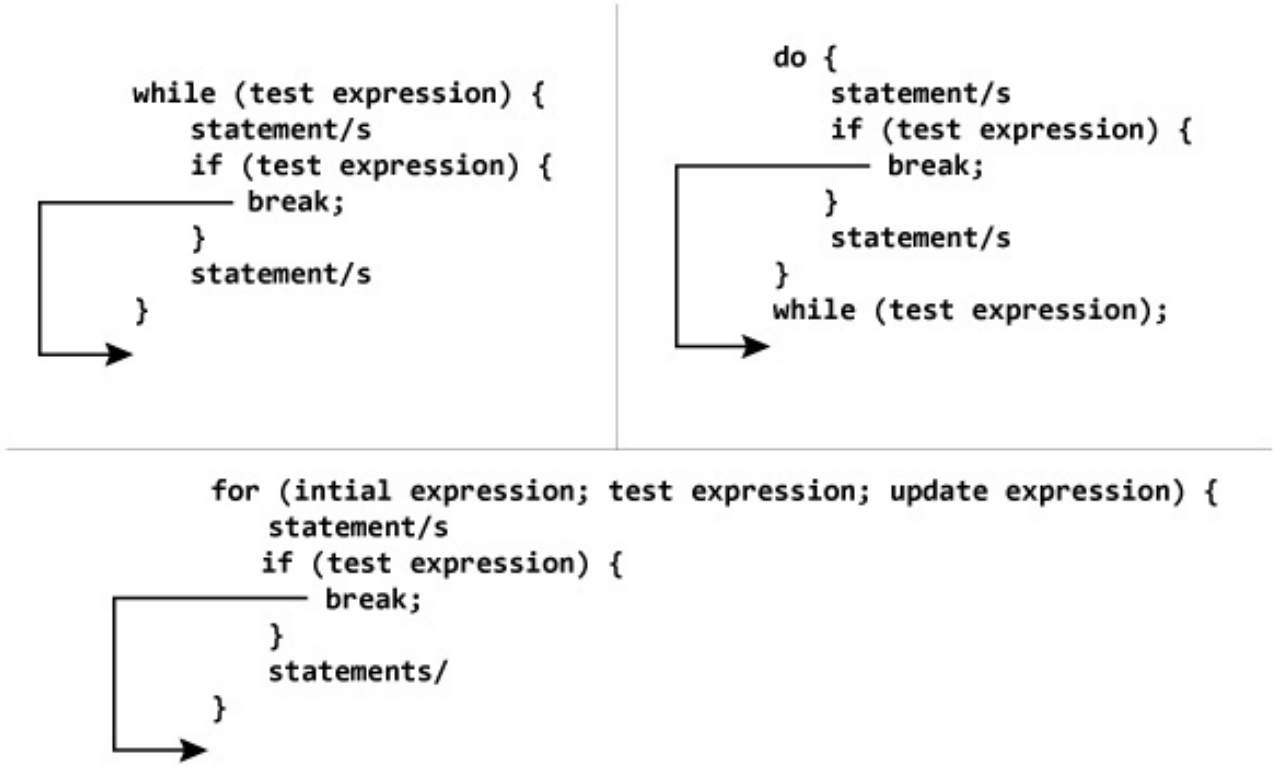
DO .. WHILE statement, program example

```
#include <stdio.h>
int main() {
   int x;
   x = 0;
   do {
      printf("Hello, world!\n");
   } while ( x != 0 );
   getchar();
}
```

Unconditional branching statements

BREAK operator, general form

Picture original source http://www.programiz.com/c-programming/c-break-continue-statement.htm



NOTE: The break statment may also be used inside body of else statement.

Unconditional branching statements

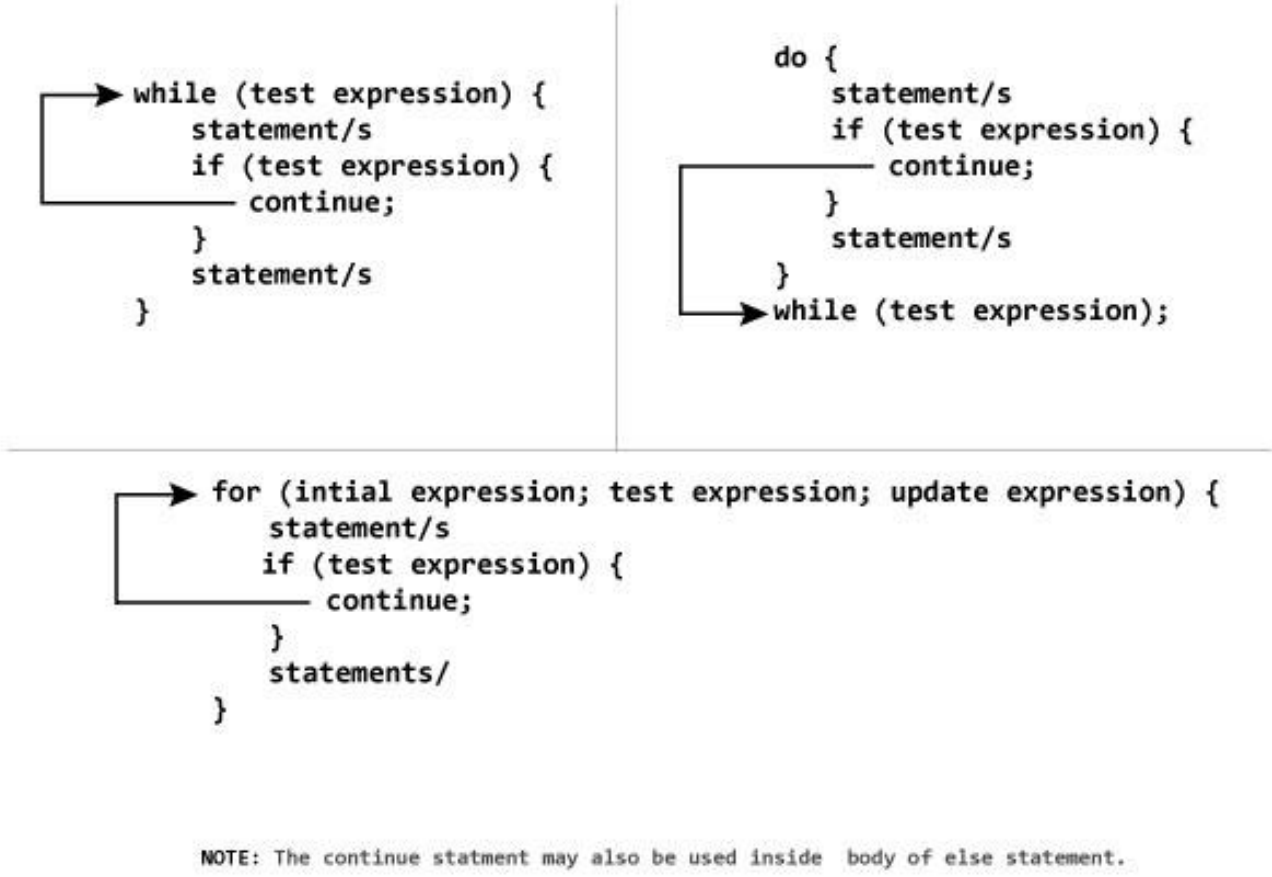
BREAK operator, program example

```
#include <stdio.h>
int main() {
   int i;
   i = 0;
   while ( i < 20 ) {
      i++;
      if (i == 10) break;
   }
   return 0;
}
```

Unconditional branching statements

CONTINUE operator, general form

Picture original source http://www.programiz.com/c-programming/c-break-continue-statement.htm



NOTE: The continue statment may also be used inside body of else statement.

Unconditional branching statements

CONTINUE operator, program example

```
#include <stdio.h>
int main() {
   int i = 0;
   while ( i < 20 ) {
      i++;
      continue;
      printf("Nothing to see\n");
   }
   return 0;
}
```

Unconditional branching statements

GOTO operator, general form

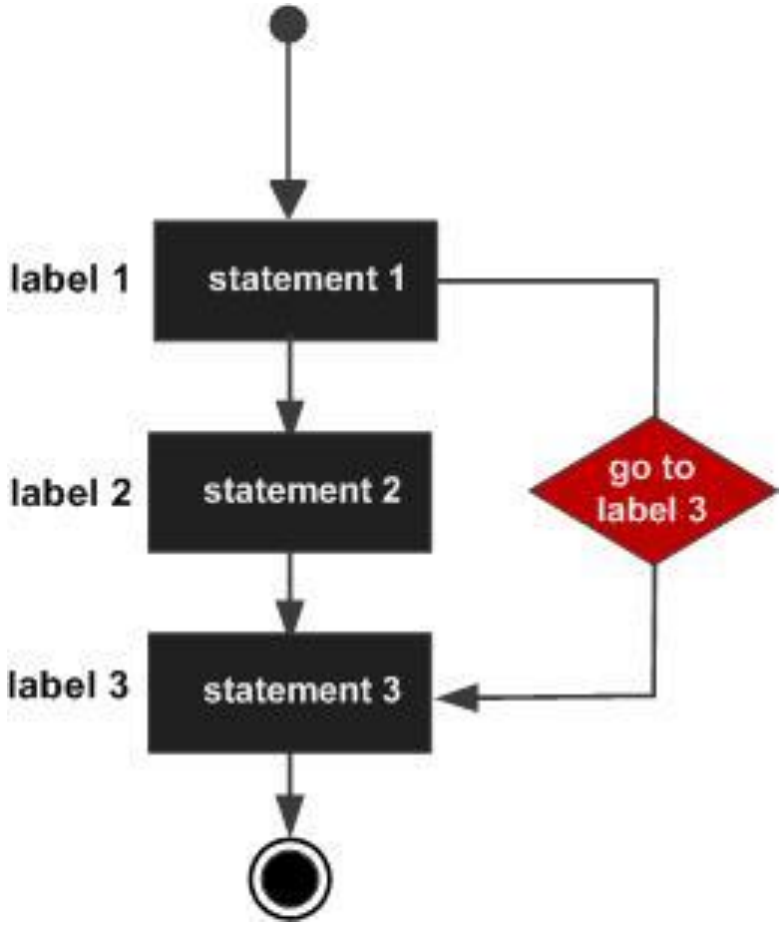
Picture original source http://www.programiz.com/c-programming/c-goto-statement.htm

General form:

```
goto label;
...
label: statement;
```

Example:

```
...
goto a;
...
a: b = b*2;
...
```



Unconditional branching statements

GOTO operator, program example (1/2)

```
#include <stdio.h>
int main () {
   int a = 10;
   LOOP:
   do {
      if( a == 15) {
```

Unconditional branching statements

GOTO operator, program example (2/2)

```
    /* skip the iteration */
        a = a + 1;
        goto LOOP;
      }
      printf("Value of a: %d\n", a);
      a++;
    } while ( a < 20 );
    return 0;
}
```

2. Input and output in programming language C

BUILT IN INPUT AND OUTPUT FUNCTIONS

Output functions

printf()

printf() – outputs the formatted text on the screen

- *General form:*
  `int printf(const char *format, ...),`
  where format may consist of such tags as:
  %[flags][width][.precision][length]specifier
- *Example:*

```
for( int ch = 75 ; ch <= 100; ch++ ) {
   printf("ASCII value = %d, Character = %c\n", ch, ch);
}
```

```
ASCII value = 75, Character = K
ASCII value = 76, Character = L
ASCII value = 77, Character = M
ASCII value = 78, Character = N
ASCII value = 79, Character = O
```

Output functions

sprintf()

sprintf() – outputs the formatted text into the string *str*

- *General form:*

```
int sprintf(char *str, const char *format,
...),
```

where format may consist of such tags as:

%[flags][width][.precision][length]specifier

- *Example:*

```
char str[80];
sprintf(str, "Value of Pi = %f", M_PI);
puts(str);
```

Output functions (1/2)

specifier

Specifier	Description
c	Character
d or i	Integer number
e	Scientific form of number with small e
E	Scientific form of number with capital E
f	Floating point number
o	Octal number

Output functions (2/2)

specifier

Specifier	Description
s	Character string
u	Unsigned integer number
x	Hexadecimal number with small letters (a..f)
X	Hexadecimal number with capital letters (A..F)
p	Pointer physical address
%	Character “%”

Output functions

flag

Flag	Description
-	Left alignment
+	Sign in front of number (“+” for positive and “-” for negative numbers)
#	Is used with specifiers o, x or X to add 0, 0x or 0X in front of appropriate number. Is used with e, E or f to output point even if value is integer
0	Fills free positions with zeros (if there are such)

Output functions

width

Width	Description
(number)	Minimum number of characters or digits on the screen for the value of variable
*	The width is not specified in the format string, but as an additional integer value argument preceding the argument that has to be formatted

Output functions

precision

Precision	Description
.number	For integer specifiers (d, i, o, u, x, X): precision specifies the minimum number of digits to be written. If the value to be written is shorter than this number, the result is padded with leading zeros. A precision of 0 means that no character is written for the value 0. For e, E and f specifiers: this is the number of digits to be printed after decimal point. For s: this is the maximum number of characters to be printed
.*	The precision is not specified in the format string, but as an additional integer value argument preceding the argument that has to be formatted

Output functions

length

Length	Description
h	Is interpreted as short int or unsigned short int (i, d, o, u, x and X)
l	Is interpreted as long int or unsigned long int (i, d, o, u, x and X), and as wide character (> 8 bites) string (c and s)
L	Is interpreted as long double (e, E, f)

Examples (1/2)

```
#include <stdio.h>

int main() {
   printf ("Characters: %c %c \n", 'a', 65);
   printf ("Decimals: %d %ld\n", 1977, 650000L);
   printf ("Preceding
            with blanks:
            %10d \n", 1977);
   printf ("Preceding with zeros: %010d \n",
            1977);
```

```
Characters: a A
Decimals: 1977 650000
Preceding with blanks:       1977
Preceding with zeros: 0000001977
```

Examples (2/2)

```
    printf ("Some different radices: %d %x %o %#x
             %#o \n", 100, 100, 100, 100, 100);
    printf ("floats: %4.2f %+.0e %E \n", 3.1416,
             3.1416, 3.1416);
    printf ("Width trick: %*d \n", 5, 10);
    printf ("%s \n", "A string");
    return 0;
}
```

```
Some different radices: 100 64 144 0x64 0144
floats: 3.14 +3e+000 3.141600E+000
Width trick:    10
A string
```

Output functions

putchar()

putchar() – outputs a character on the screen

- *General form:*

```
int putchar(int char)
```

- *Example:*

```
for(char ch = 'A'; ch <= 'Z'; ch++) {
  putchar(ch);
}
```

```
ABCDEFGHIJKLMNOPQRSTUVWXYZ
```

Output functions

puts()

puts() – outputs a string on the screen

- *General form:*

```
int puts(const char *str)
```

- *Example:*

```
char str[15];
strcpy(str, "tutorialspoint");
puts(str);
```

```
tutorialspoint
```

Input functions

scanf()

scanf() – reads the formatted text from the screen

- *General form:*
  `int scanf(const char *format, ...),`
  where format may consist of such tags as:
  %[*][width][length]specifier
- *Example:*

```
char str[20];
printf("Enter name: ");
scanf("%s", &str);
```

Input functions

sscanf()

sscanf() – reads the formatted text from a string *str*

- *General form:*

```
int sscanf(const char *str, const char
*format, ...),
```

where format may consist of such tags as: **%[*][width][length]specifier**

- *Example:*

```
strcpy(dtm, "Saturday March 25 1989");
sscanf(dtm, "%s %s %d %d", weekday, month,
        &day, &year);
printf("%s %d, %d = %s\n", month, day, year,
        weekday);
```

```
March 25, 1989 = Saturday
```

Input functions

Format

Argument	Description
*	Defines that data will be taken from the input, but will no be stored in variable
width	Defines the maximum number of characters that can be taken from the input
length	Defines the data type (similar to length in printf() function)
specifier	Defines how to interpret the text (similar to specifiers in printf() function)

Examples (1/2)

```
#include <stdio.h>
int main () {
   char str [80];
   int i;
   printf ("Enter your family name: ");
   scanf ("%79s",str);
   printf ("Enter your age: ");
   scanf ("%d",&i);
```

```
Enter your family name: Soulie
Enter your age: 29
```

Examples (2/2)

```
  printf ("Mr. %s , %d years old.\n",str,i);
  printf ("Enter a hexadecimal number: ");
  scanf ("%x",&i);
  printf ("You have entered %#x (%d).\n",i,i);
  return 0;
}
```

```
Mr. Soulie , 29 years old.
Enter a hexadecimal number: ff
You have entered 0xff (255).
```

Input functions

getchar()

getchar() – reads a character form the screen

- *General form:*

```
int getchar(void)
```

- *Example:*

```
char c;
printf("Enter character: ");
c = getchar();
```

Input functions

**gets()*

***gets()** – reads a string from the screen

- *General form:*

```
char *gets(char *str)
```

- *Example:*

```
char str[50];
printf("Enter a string : ");
gets(str);
```

3. Practical work #2

A SIMPLE PROGRAM IN PROGRAMMING LANGUAGE C

A simple program in programming language C

Create a program that performs the following actions:

1. Asks a user to input 2 integer numbers.

2. If the sum of these numbers (S) is less than 20:
   - counts how many odd numbers are between 0 and S;
3. If the sum of these numbers (s) is grater than or equal to 20:
   - counts how many numbers divisible by 7 are between 20 and S.